

Operating Systems: Concurrency, Synchronization & Deadlocks

Academic Lecture Notes & Study Companion

Department of Computer Science • Systems & Networking Division

Core Conceptual Takeaways

- **Isolation vs. Sharing:** Processes run in dedicated, hardware-isolated virtual address spaces with distinct page tables; threads share virtual memory, code segment, and heap inside a single enclosing process.
- **Mutual Exclusion Requirement:** Unsynchronized concurrent access to shared mutable data introduces non-deterministic race conditions. Critical sections must be executed atomically.
- **The Deadlock Theorem:** A deadlock can only occur if and only if all **four Coffman conditions** (Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait) hold simultaneously.

1 Process Isolation vs. Thread Concurrency

Modern computing systems rely on concurrency to execute multiple units of computation simultaneously or via interleaved timeslices.

Process vs. Thread Architecture

Process: An independent, executing instance of a program. It possesses its own isolated virtual address space, file descriptor table, register state, user credentials, and memory mappings managed by the OS page tables. Context switching between distinct processes requires invalidating or updating translation lookaside buffer (TLB) entries.

Thread: The smallest schedulable unit of CPU execution inside a process (often called a *lightweight process*). All peer threads within a process share the exact same virtual memory space, global data segment, and dynamic heap, but each maintains its own private stack and hardware program counter (PC).

High-Yield Mnemonic

P-T-A-H: Processes own isolated Address spaces; Threads share Heap & handles.

2 Race Conditions & The Critical Section Problem

When multiple execution threads read and write shared data without synchronization, the final outcome depends on thread scheduling order, causing a **Race Condition**.

2.1 Properties of a Correct Solution

Any valid critical section synchronization mechanism must fulfill three fundamental requirements:

1. **Mutual Exclusion:** If thread T_i is executing in its critical section, no other thread may enter the critical section.
2. **Progress:** If no thread is in its critical section and threads wish to enter, selection cannot be postponed indefinitely.
3. **Bounded Waiting:** There must exist a bound on the number of times other threads can enter their critical sections after a thread has made a request.

3 Synchronization Primitives

Operating systems and hardware architectures provide several primitives to enforce atomic coordination:

- **Mutex (Mutual Exclusion Lock):** A binary synchronization tool featuring *ownership semantics*. Only the exact thread that acquired the mutex is permitted to release it.
- **Semaphore:** A signaling variable introduced by Edsger Dijkstra. A semaphore S is initialized to an integer counter and accessed through two atomic primitives:

$$\begin{aligned} \text{wait}(S) / P(S) : & \quad \text{while } S \leq 0 \text{ do wait; } S \leftarrow S - 1 \\ \text{signal}(S) / V(S) : & \quad S \leftarrow S + 1 \end{aligned}$$

Unlike mutexes, semaphores have no thread ownership and can signal between producer and consumer threads.

- **Spinlock:** A locking primitive where threads waiting for the lock poll continuously in a CPU loop (*busy-waiting*). While efficient for very short wait times on multi-core systems, spinlocks waste CPU cycles on single-core uniprocessors.

4 The Deadlock Dilemma & The 4 Coffman Conditions

A **Deadlock** is a permanent systemic impasse where every process in a set is blocked waiting for an event that only another blocked process in the set can cause.

The 4 Necessary Coffman Conditions

All four conditions must hold simultaneously for a deadlock to arise:

1. **Mutual Exclusion:** At least one resource must be held in a non-shareable mode.
2. **Hold and Wait:** A process must be holding at least one resource and simultaneously waiting to acquire additional resources held by other processes.
3. **No Preemption:** Resources cannot be forcibly confiscated from a process; they can only be released voluntarily by the holding process upon task completion.
4. **Circular Wait:** A closed chain of processes $\{P_0, P_1, \dots, P_n\}$ exists such that P_0 waits for a resource held by P_1 , and P_n waits for a resource held by P_0 .

High-Yield Mnemonic

M-H-N-C: Mutual Exclusion, **H**old and Wait, **N**o Preemption, **C**ircular Wait.

5 Deadlock Prevention & Dijkstra's Banker's Algorithm

System designers employ two distinct mathematical strategies to eliminate deadlocks:

Deadlock Prevention: Restricts resource requests such that at least one of the four Coffman conditions can never hold. Enforcing a **strict total ordering** on all resource acquisitions permanently breaks the Circular Wait condition.

Deadlock Avoidance (Banker's Algorithm): Allows dynamic resource requests but evaluates current allocation vectors against maximum claim matrices before granting an allocation. A request is granted if and only if the system transitions into a guaranteed **Safe State** (a state from which there exists an execution sequence allowing every process to finish without deadlocking).